Unidad 4:

PLANIFICACIÓN DE TAREAS Y SEGURIDAD EN BBDD

FICHA TÉCNICA DE LA UNIDAD

Unidad Didáctica	Unidad 4: Planificación de Tareas y Seguridad en Bases de Datos
Resultados de Aprendizaje	**RA3:** Automatiza tareas de administración del sistema gestor, realizando guiones y programando tareas. **RA6:** Asegura la disponibilidad y recuperación de la información aplicando políticas de copias de seguridad y replicación.
Criterios de Evaluación	**CE 3.b:** Se ha configurado el programador de tareas del SGBD para programar la ejecución periódica de tareas. **CE 6.a:** Se han planificado y ejecutado copias de seguridad lógicas (exportación de esquemas) y físicas. **CE 6.b:** Se han diseñado estrategias de recuperación de desastres ante fallos lógicos o físicos del almacenamiento. **CE 6.c:** Se ha configurado el modo de recuperación del SGBD (logs de transacciones activos, archivado). **CE 6.e:** Se ha verificado la integridad de las copias de seguridad mediante restauraciones automatizadas.
Tiempo	12 horas (equivalente a 4 semanas de clase en régimen presencial)

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

2. PLANIFICACIÓN DE TAREAS AUTOMÁTICAS MEDIANTE EVENTOS

    2.1. Introducción al Planificador de Eventos (Event Scheduler)

    2.2. Verificación y Activación del Planificador

2.3. Sintaxis de Creación de Eventos (CREATE EVENT)

2.4. Ejemplos Prácticos de Creación de Eventos

2.5. Gestión de Eventos: Modificación, Activación, Desactivación y Eliminación

2.6. Visualización y Monitorización de Eventos en el Sistema

3. GESTIÓN DE LA SEGURIDAD DE LOS DATOS

3.1. Clasificación y Tipos de Copias de Seguridad (Backup)

3.2. Directorio de Datos de MySQL y Estructura de Archivos

3.3. Copias de Seguridad Físicas o Binarias

3.4. Copias de Seguridad Lógicas o Textuales

3.4.1. Sentencias SQL de Importación/Exportación

3.4.2. Uso Avanzado de mysqldump

3.5. Restauración de Copias de Seguridad Lógicas

3.6. Gestión de Backups mediante Interfaces Gráficas

4. BUENAS PRÁCTICAS

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

El papel de un Administrador de Sistemas de Bases de Datos (DBA) y de un Administrador de Sistemas (SysAdmin) implica garantizar la continuidad del negocio mediante políticas de alta disponibilidad, la programación inteligente de tareas recurrentes y el diseño de planes de contingencia ante pérdidas de datos.

Esta unidad didáctica aborda de forma integral la automatización interna de tareas de mantenimiento mediante el planificador nativo de eventos de MySQL y profundiza en las bases teóricas y herramientas prácticas para salvaguardar y restaurar bases de datos en entornos de producción.

Esta unidad se relaciona directamente con el módulo de *Implantación de Sistemas Operativos* (1º ASIR) al administrar y interactuar con servicios y rutas del sistema operativo Linux host (como directorios /var/lib/mysql/ o permisos de log de errores), y sienta las bases lógicas de seguridad y recuperación necesarias para el módulo de *Implantación de Aplicaciones Web* (2º ASIR), donde las bases de datos mantenidas actuarán como el motor de persistencia back-end de servidores de producción corporativos.

2. PLANIFICACIÓN DE TAREAS AUTOMÁTICAS MEDIANTE EVENTOS

2.1. Introducción al Planificador de Eventos (Event Scheduler)

Un evento en MySQL es un objeto de base de datos que encapsula sentencias SQL destinadas a ejecutarse de forma automática en un momento de tiempo programado o a intervalos periódicos.

En entornos Linux, los eventos de MySQL sustituyen de manera óptima a las herramientas del programador de sistema operativo (*cron jobs*). Ejecutar las tareas de mantenimiento directamente en el SGBD reduce la sobrecarga de conexiones TCP/IP de scripts externos de mantenimiento y mejora la portabilidad de los modelos de base de datos entre distintos entornos.

2.2. Verificación y Activación del Planificador

El planificador de eventos funciona como un hilo daemon en el servidor. El estado del planificador se controla mediante la variable de sistema global event_scheduler. Sus valores pueden ser:

- **OFF (o 0):** El hilo del planificador está parado. (Valor inicial por defecto en muchas distribuciones).
- **ON (o 1):** El planificador está activo y procesando la cola de tareas automáticas.
- `DISABLED: El hilo está deshabilitado físicamente y no se puede activar dinámicamente en caliente.`

Verificación:

```
SHOW VARIABLES LIKE 'event_scheduler';
-- O alternativamente mediante consulta directa:
SELECT @@global.event_scheduler;
```

Activación Dinámica (en caliente):

```
SET GLOBAL event_scheduler = ON;
```

Configuración Permanente:

Para evitar que se desactive tras un reinicio del servicio, se añade al fichero de configuración (/etc/mysql/mysql.conf.d/mysqld.cnf o my.ini):

```
[mysqld]
event_scheduler=ON
```

2.3. Sintaxis de Creación de Eventos (CREATE EVENT)

```
CREATE
[DEFINER = { user | CURRENT_USER }]
EVENT
[IF NOT EXISTS] event_name
ON SCHEDULE schedule
[ON COMPLETION [NOT] PRESERVE]
[ENABLE | DISABLE | DISABLE ON SLAVE]
[COMMENT 'string']
DO event_body;
schedule: {
AT timestamp [+ INTERVAL interval] ...
| EVERY interval
[STARTS timestamp [+ INTERVAL interval] ...]
[ENDS timestamp [+ INTERVAL interval] ...]
}
```

Cláusulas Clave:

- **DEFINER:** Define el nivel de privilegios y el contexto de seguridad con el que se ejecutará el cuerpo del evento.
- **ON SCHEDULE:** Define cuándo actúa. AT se emplea para ejecuciones únicas, mientras que EVERY planifica intervalos cíclicos.
- **ON COMPLETION PRESERVE:** Al activarlo, el evento no se elimina del diccionario de datos una vez haya concluido su vigencia temporal o su ejecución única (por defecto es NOT PRESERVE y el SGBD lo destruye de forma automática).
- **DISABLE ON SLAVE:** Evita que el evento se replique y ejecute de manera concurrente en servidores esclavos de lectura, limitando su ejecución

exclusivamente al servidor maestro.

2.4. Ejemplos Prácticos de Creación de Eventos

Ejemplo 1: Evento de Ejecución Única (AT)

Un evento que se ejecutará de forma única pasados exactamente 5 minutos desde su creación para depurar registros huérfanos. Se mantendrá registrado en el sistema tras concluir.

```
CREATE EVENT IF NOT EXISTS ev_depurar_huerfanos_unico
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 5 MINUTE
ON COMPLETION PRESERVE
COMMENT 'Limpia registros de transacciones huérfanas una única
vez.'
DO
DELETE FROM tienda.pedidos_temporales
WHERE completado = 0 AND fecha_creacion < NOW() - INTERVAL 1
HOUR;
```

Ejemplo 2: Evento Periódico (EVERY) con Bloque de Sentencias y delimitador

Para programar un evento que ejecute múltiples operaciones complejas, se debe encapsular la lógica en un bloque estructurado (BEGIN ... END) y redefinir el delimitador temporal de fin de instrucción:

```
DELIMITER $$
CREATE EVENT IF NOT EXISTS ev_mantenimiento_mensual_ventas
ON SCHEDULE EVERY 1 MONTH
STARTS (CURRENT_TIMESTAMP + INTERVAL 1 DAY) -- Empieza mañana
COMMENT 'Agrupa ventas del mes pasado en histórico y vacía tablas
transaccionales.'
DO
BEGIN
-- 1. Copiar históricos de facturas
INSERT INTO historico.facturas_antiguas
SELECT * FROM tienda.facturas
WHERE fecha_emision < DATE_SUB(NOW(), INTERVAL 1 MONTH);
-- 2. Eliminar de la tabla caliente de producción
DELETE FROM tienda.facturas
WHERE fecha_emision < DATE_SUB(NOW(), INTERVAL 1 MONTH);
```

```
END$$
DELIMITER ;
```

2.5. Gestión de Eventos: Modificación, Activación, Desactivación y Eliminación

Modificación de Eventos (ALTER EVENT)

Permite cambiar las propiedades de planificación, estado o comportamiento de un evento existente de forma dinâmica sin afectar al diccionario de datos:

```
-- Desactivar temporalmente el evento de mantenimiento
ALTER EVENT ev_mantenimiento_mensual_ventas DISABLE;
-- Modificar su planificación para que se ejecute a partir de hoy en
tiempo real
ALTER EVENT ev_mantenimiento_mensual_ventas
ON SCHEDULE EVERY 1 MONTH STARTS NOW()
ENABLE;
```

Borrado de Eventos (DROP EVENT)

Elimina definitivamente un evento del servidor de bases de datos:

```
DROP EVENT IF EXISTS ev_depurar_huerfanos_unico;
```

2.6. Visualización y Monitorización de Eventos en el Sistema

Listado de Eventos Activos en el Esquema actual

```
SHOW EVENTS;
```

Consulta del Diccionario de Datos del Sistema:

La vista de catálogo INFORMATION_SCHEMA.EVENTS almacena metadatos críticos para auditar el rendimiento y planificación de tareas (como cuándo fue la última ejecución real):

```
SELECT EVENT_SCHEMA, EVENT_NAME, STATUS, LAST_EXECUTED FROM
INFORMATION_SCHEMA.EVENTS WHERE EVENT_SCHEMA = 'tienda';
```

Logs de errores y estado del planificador:

El inicio, parada o fallos en tiempo de ejecución del cuerpo de un evento se registran en el fichero de errores de MySQL en disco. En Linux, se puede auditar con:

```
sudo cat /var/log/mysql/error.log | grep -i "Event"
```

3. GESTIÓN DE LA SEGURIDAD DE LOS DATOS

3.1. Clasificación y Tipos de Copias de Seguridad (Backup)

Tipo de Clasificación	Variantes	Explicación Técnica y Comportamiento
Disponibilidad del Servicio	**En Frío (*Offline*)**	Se detiene el servidor de base de datos. No hay transacciones. Garantiza consistencia absoluta de archivos pero introduce *downtime*.
	En Caliente (*Online*)	El servidor sigue activo. Permite lecturas y escrituras concurrentes. Requiere motores transaccionales (InnoDB) para evitar incoherencias.
Formato del Respaldo	**Física o Binaria**	Copia directa de archivos de datos estructurados de disco. Velocidad de restauración y copia muy rápida. Poca portabilidad.
	Lógica o Textual	Exporta la información a sentencias SQL legibles (CREATE, INSERT) o texto plano delimitado. Excelente portabilidad.

Tipo de Clasificación	Variantes	Explicación Técnica y Comportamiento
		Lentitud en restauración.
Alcance de la Información	**Completa (*Full*)**	Respaldo del 100% de esquemas, vistas, índices y datos del servidor.
	Incremental	Respalda únicamente los bloques de datos modificados desde el último backup realizado.

3.2. Directorio de Datos de MySQL y Estructura de Archivos

En sistemas Linux, MySQL utiliza el siguiente directorio por defecto para el almacenamiento físico: /var/lib/mysql/

Dentro de esta ruta se crea un directorio para cada base de datos. Los archivos que componen una tabla física dependen directamente de su motor de almacenamiento:

- **Archivos MyISAM:**
  - .frm: Definición y metadatos de la estructura de la tabla (en desuso en MySQL 8.0).
  - .MYD (*MyData*): Almacena los registros de datos físicos de la tabla.
  - .myi (*MyIndex*): Almacena los índices estructurados de MyISAM.
- **Archivos InnoDB:**
  - .ibd: Almacena conjuntamente los datos lógicos y los índices de una tabla de tipo InnoDB (siempre que la variable global de motor innodb_file_per_table esté activada).

3.3. Copias de Seguridad Físicas o Binarias

Operación de copia física en caliente de tablas MyISAM:

```
-- 1. Cerrar tablas e impedir escrituras externas bloqueando para
lectura
FLUSH TABLES producto FOR EXPORT;
```

(En este punto, el SysAdmin copia de forma segura los archivos .MYD y .myi de la ruta física /var/lib/mysql/tienda/ a un directorio externo de almacenamiento).

```
-- 2. Liberar el bloqueo para restaurar la operatividad en caliente
del servidor
UNLOCK TABLES;
```

Operación de restauración física de tablas InnoDB (DISCARD / IMPORT):

Dado que InnoDB gestiona su propio espacio de tablas (*tablespace*), la restauración física de un archivo .ibd de forma individual requiere el siguiente procedimiento:

```
-- 1. Desasociar el espacio de tablas lógico actual en MySQL
ALTER TABLE tienda.producto DISCARD TABLESPACE;
```

(El SysAdmin copia de forma manual el archivo físico producto.ibd de respaldo al directorio /var/lib/mysql/tienda/ y le asigna la propiedad de usuario necesaria: chown mysql:mysql).

```
-- 2. Importar el nuevo espacio de tablas restaurado
ALTER TABLE tienda.producto IMPORT TABLESPACE;
```

3.4. Copias de Seguridad Lógicas o Textuales

3.4.1. Sentencias SQL de Importación/Exportación

Exportación de Datos (SELECT ... INTO OUTFILE)

Vuelca los datos de una consulta en formato de texto plano estructurado. Requiere el uso del directorio seguro de MySQL (auditable mediante la variable global secure_file_priv):

```
SELECT codigo, nombre, cantidad_en_stock INTO OUTFILE
'/var/lib/mysql-files/fabricante_backup.txt' FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n' FROM tienda.fabricante;
```

Importación de Datos (LOAD DATA)

Carga un archivo de texto estructurado de forma inmediata en una tabla previamente definida:

```
LOAD DATA INFILE '/var/lib/mysql-files/fabricante_backup.txt' INTO
TABLE tienda.fabricante_copia FIELDS TERMINATED BY ',' LINES TERMINATED
BY '\n';
```

3.4.2. Uso Avanzado de mysqldump

La herramienta principal de administración de copias lógicas en consola es mysqldump. Su sintaxis y modos de uso principales son:

A. Exportar una única tabla de una base de datos específica:

```
mysqldump -u root -p tienda producto >
backup_producto_tienda.sql
```

B. Exportar una o varias bases de datos completas (incluye sentencias CREATE DATABASE):

```
mysqldump -u root -p --databases tienda biblioteca >
backup_sistemas.sql
```

C. Exportar toda la instancia de MySQL (todas las bases de datos de usuario y sistema):

```
mysqldump -u root -p --all-databases >
backup_completo_servidor.sql
```

Parámetros Útiles de Optimización de mysqldump:

- --no-data: Copia únicamente la estructura de las tablas (crea el esquema DDL), omitiendo la inserción de datos.

- --no-create-info: Copia exclusivamente los datos en sentencias INSERT (DML), útil para poblar tablas existentes sin alterarlas.
- --single-transaction: Permite realizar copias consistentes en caliente de tablas **InnoDB** sin bloquear las transacciones ni escrituras concurrentes del servidor.

3.5. Restauración de Copias de Seguridad Lógicas

Para ejecutar las sentencias SQL contenidas en un archivo de volcado lógico de manera externa, se emplea el cliente estándar de terminal de MySQL y el operador de redirección:

Opción A: Desde el terminal del sistema operativo (Redirección externa <)

```
-- Si el backup tiene sentencias 'CREATE DATABASE' integradas:
mysql -u root -p < backup_sistemas.sql
-- Si el backup no las tiene, se especifica la base de datos de
destino:
mysql -u root -p tienda < backup_producto_tienda.sql
```

Opción B: Desde el intérprete de comandos interno de MySQL (source)

```
mysql> USE tienda;
mysql> SOURCE /var/lib/mysql-files/backup_producto_tienda.sql;
```

3.6. Gestión de Backups mediante Interfaces Gráficas

- `MySQL Workbench: Ofrece el módulo Data Export y Data Import que encapsulan el funcionamiento de la utilidad de consola mysqldump bajo una interfaz gráfica de pestañas para exportar e importar cómodamente esquemas o carpetas autocontenidas de proyectos.`
- **phpMyAdmin:** Proporciona un entorno web de administración que permite descargar respaldos comprimidos de bases de datos directamente mediante el navegador, ajustando parámetros de exportación DDL/DML, y subir ficheros .sql para su ejecución mediante el cargador integrado.

4. BUENAS PRÁCTICAS

Como administradores de sistemas gestores de bases de datos, podemos automatizar de manera muy ágil rutinas de mantenimiento complejas combinando de forma segura ambas partes de la materia:

1. **Purga periódica de Logs transaccionales:** Diseñar un evento diario que vacíe tablas de errores transaccionales volátiles de poco valor histórico antes de que se ejecute la copia lógica de seguridad (mysqldump). De esta forma, reducimos el peso en disco del backup y optimizamos el almacenamiento.

2. **Historial de Operaciones de Backup en Eventos:** Al planificar tareas que copien tablas transaccionales a tablas temporales locales (por ejemplo, equipos a equipos_temp), es recomendable registrar una entrada en un log de eventos propio que documente la hora exacta y el usuario que disparó la operación para que esta información quede recogida y salvaguardada en la auditoría general del sistema.